

1 Syntax

$$\begin{aligned}
\epsilon &::= \text{BEndian} \mid \text{LEndian} \\
\Diamond_u &::= \text{NEG} \mid \text{NOT} \\
\Diamond_b &::= \text{ADD} \mid \text{SUB} \mid \text{MUL} \mid \text{DIV} \mid \text{SDIV} \mid \text{MOD} \mid \text{SMOD} \\
&\quad \mid \text{SHL} \mid \text{SHR} \mid \text{SAR} \mid \text{AND} \mid \text{OR} \mid \text{XOR} \mid \text{CONCAT} \\
\Diamond_r &::= \text{EQ} \mid \text{NEQ} \mid \text{GT} \mid \text{GE} \mid \text{LT} \mid \text{LE} \\
&\quad \mid \text{SGT} \mid \text{SGE} \mid \text{SLT} \mid \text{SLE} \\
\Diamond_c &::= \text{ZeroExt} \mid \text{SignExt} \\
exp &::= \text{BinOp}(\Diamond_b, k, exp, exp, \mu) \\
&\quad \mid \text{Cast}(\Diamond_c, k, exp, \mu) \\
&\quad \mid \text{ExprList}(exp^*, \mu) \\
&\quad \mid \text{Extract}(exp, k, pos, \mu) \\
&\quad \mid \text{FuncName}(s, \mu) \\
&\quad \mid \text{ITE}(exp_c, exp_t, exp_f, \mu) \\
&\quad \mid \text{JmpDest}(\ell, \mu) \\
&\quad \mid \text{Load}(\epsilon, k, exp, \mu) \\
&\quad \mid \text{Num}(n, \mu) \\
&\quad \mid \text{PCVar}(k, name, \mu) \\
&\quad \mid \text{RelOp}(\Diamond_r, exp, exp, \mu) \\
&\quad \mid \text{TempVar}(k, id, \mu) \\
&\quad \mid \text{UnOp}(\Diamond_u, exp, \mu) \\
&\quad \mid \text{Undefined}(k, str, \mu) \\
&\quad \mid \text{Var}(k, reg_{id}, name, \mu) \\
stmt &::= \text{CJmp}(exp_c, exp_t, exp_f, \mu) \\
&\quad \mid \text{ExternalCall}(exp, \mu) \\
&\quad \mid \text{IEMark}(len, \mu) \\
&\quad \mid \text{ISMark}(len, \mu) \\
&\quad \mid \text{InterCJmp}(exp_c, exp_t, exp_f, \mu) \\
&\quad \mid \text{InterJmp}(exp, jumpKind, \mu) \\
&\quad \mid \text{Jmp}(exp, \mu) \\
&\quad \mid \text{LMark}(\ell, \mu) \\
&\quad \mid \text{Put}(exp, exp, \mu) \\
&\quad \mid \text{SideEffect}(SideEffect, \mu) \\
&\quad \mid \text{Store}(\epsilon, exp, exp, \mu)
\end{aligned}$$

k: Register Type (n-bit)

2 Abstract Domains

(Abstract Integer)	$\mathbb{I}$	$= a \cdot \text{symbol} + b \mid \perp \mid \top$
(Abstract Location)	$\mathbb{L}$	$= \text{Global}(\mathbb{Z})$ $\mid \text{Stack}(\text{function} \times \mathbb{Z})$ $\mid \text{Heap}(\text{allocsite} \times \mathbb{Z})$ $\mid \text{SymLoc}(\text{symbol} \times \mathbb{Z})$
(Abstract Type)	$\mathbb{T}$	$= \text{Value} \mid \text{Address} \mid \perp \mid \top$
(Abstract Value)	$\mathbb{V}$	$= \mathbb{I} \times 2^{\mathbb{L}} \times \mathbb{Z}_{\mathbb{T}}$
(Register Map)	$\mathbb{R}$	$= \text{reg} \rightarrow \mathbb{V}$
(Memory)	$\mathbb{M}$	$= \mathbb{L} \rightarrow \mathbb{V}$
(Type Map)	$\mathbb{T}_{\mathbb{M}}$	$= \mathbb{Z}_{\mathbb{T}} \rightarrow \mathbb{T}$
(Type Constraint)	$\mathbb{T}_{\mathbb{C}}$	$= \text{OneOf}(2^{\mathbb{Z}_{\mathbb{T}}}) \mid \text{Same}(2^{\mathbb{Z}_{\mathbb{T}}})$
(Abstract State)	$\mathbb{S}$	$= \mathbb{R} \times \mathbb{M} \times \langle \mathbb{T}_{\mathbb{M}} \times 2^{\mathbb{T}_{\mathbb{C}}} \rangle$

3 Summary Domain

(Argument Map)	$\mathbb{A}$	$= \mathbb{Z} \rightarrow \mathbb{V}$
(Update Set)	$\mathbb{U}$	$= 2^{\mathbb{L} \times \mathbb{V}}$
(Side Effect)	Δ	$= \mathbb{A} \times \mathbb{U}$
(Operand Type Map)	$\mathbb{O}$	$= \text{instr} \rightarrow \text{operand} \rightarrow \mathbb{Z}_{\mathbb{T}}$
(Function Summary)	Σ_F	$= \langle \Delta, \text{Ret} \rangle$
(Return Value of Summarize)	Σ	$= \langle \Sigma_F, \mathbb{O}, \mathbb{T}_{\mathbb{M}} \rangle$

4 Auxiliary Functions

$$\text{Loc}(n) = \begin{cases} \{\text{Global}(n)\} & n \in \text{GlobalRegion} \\ \emptyset & \text{otherwise} \end{cases}$$

$$\text{locs}(\langle i, L, t \rangle) = L$$

$$\text{type}(\langle i, L, t \rangle) = T_M(t)$$

$$\text{new}(T_M) = \min\{t \in \mathbb{Z}_{\mathbb{T}} \mid t \notin \text{dom}(T_M)\}$$

5 Type Join

$$\tau_1 \sqcup \tau_2 = \begin{cases} \tau_2 & \tau_1 = \perp \\ \tau_1 & \tau_2 = \perp \\ \tau_1 & \tau_1 = \tau_2 \\ \top & \text{otherwise} \end{cases}$$

6 Value Join

$$\langle i_1, L_1, \langle t_1, tc_1 \rangle \rangle \sqcup \langle i_2, L_2, \langle t_2, tc_2 \rangle \rangle = \langle i_1 \sqcup i_2, L_1 \cup L_2, \langle t, tc_1 \cup tc_2 \cup \text{Same}(t_1, t_2) \rangle \rangle$$

7 Type Update

$$\text{update}_T(S, v, \tau \mid *) \Downarrow S'$$

$$\frac{}{\text{update}_T(S, v, *) \Downarrow S}$$

$$\frac{S = \langle R, M, \langle T_M, T_C \rangle \rangle \quad v = \langle i, L, t \rangle}{\text{update}_T(S, v, \tau) \Downarrow \langle R, M, \langle T_M[t \mapsto T_M(t) \sqcup \tau], T_C \rangle \rangle}$$

7.1 BinOp

$$\text{updateBinOpT}(S, \diamond_b, v_1, v_2, \tau \mid *) \Downarrow S'$$

$$\frac{\diamond_b = \text{ADD} \quad v_1 = \langle i_1, L_1, t_1 \rangle \quad v_2 = \langle i_2, L_2, t_2 \rangle \quad S = \langle R, M, \langle T_M, T_C \rangle \rangle}{\text{updateBinOpT}(S, \diamond_b, v_1, v_2, \text{Address}) \Downarrow \langle R, M, \langle T_M, T_C \cup \{\text{OneOf}(t_1, t_2)\} \rangle \rangle}$$

$$\frac{\diamond_b = \text{ADD} \quad \text{update}_T(S, v_1, \tau) \Downarrow S_1 \quad \text{update}_T(S_1, v_2, \tau) \Downarrow S_2}{\text{updateBinOpT}(S, \diamond_b, v_1, v_2, \tau) \Downarrow S_2}$$

$$\frac{\diamond_b = \text{SUB} \quad \text{update}_T(S, v_1, \text{Address}) \Downarrow S_1 \quad \text{update}_T(S_1, v_2, \text{Value}) \Downarrow S_2}{\text{updateBinOpT}(S, \diamond_b, v_1, v_2, \text{Address}) \Downarrow S_2}$$

$$\text{updateBinOpT}(S, \text{AND}, v_1, v_2, \tau) \Downarrow \text{update}_T(S, v_2, \text{Value})$$

$$\frac{\text{update}_T(S, v_1, \text{Value}) \Downarrow S_1 \quad \text{update}_T(S_1, v_2, \text{Value}) \Downarrow S_2}{\text{updateBinOpT}(S, \diamond_b, v_1, v_2, \tau) \Downarrow S_2}$$

8 Expr Evaluation

$$eval(S, expr) \Downarrow \langle R \times S \rangle$$

8.1 BinOp

$$\frac{eval(S, e_1) \Downarrow \langle v_1, S_1 \rangle \quad eval(S_1, e_2) \Downarrow \langle v_2, S_2 \rangle \quad updateBinOpT(S_2, \diamond_b, v_1, v_2, *) \Downarrow S_3 \quad evalBinOp(\diamond_b, k, v_1, v_2, S_3) \Downarrow \langle v, S' \rangle}{eval(S, BinOp(\diamond_b, k, e_1, e_2, \mu)) \Downarrow \langle v, S' \rangle}$$

8.2 Cast

$$\frac{eval(S, e) \Downarrow \langle \langle i, L, t \rangle, S_1 \rangle \quad v = \langle cast_{\diamond_c, k}(i), L, t \rangle}{eval(S, Cast(\diamond_c, k, e, \mu)) \Downarrow \langle v, S_1 \rangle}$$

8.3 ExprList

$$\frac{eval(S, e_0) \Downarrow \langle v_0, S_1 \rangle \quad eval(S_1, e_1) \Downarrow \langle v_1, S_2 \rangle \quad \cdots \quad eval(S_n, e_n) \Downarrow \langle v_n, S' \rangle}{eval(S, ExprList(e_0, \dots, e_n, \mu)) \Downarrow \langle \langle v_0, \dots, v_n \rangle, S' \rangle}$$

8.4 Extract

$$\frac{eval(S, e) \Downarrow \langle \langle i, L, t \rangle, S_1 \rangle \quad updateT(S_1, \langle i, L, t \rangle, Value) \Downarrow S_2 \quad v = \langle extract(i, k, pos), L, t \rangle}{eval(S, Extract(e, k, pos, \mu)) \Downarrow \langle v, S_2 \rangle}$$

8.5 FuncName

$$\frac{S = \langle R, M, \langle T_M, T_C \rangle \rangle \quad t = new(T_M)}{eval(S, FuncName(s, \mu)) \Downarrow \langle \langle \top, \{SymLoc(s, 0)\}, t \rangle, \langle R, M, \langle T_M[t \mapsto Address], T_C \rangle \rangle \rangle}$$

8.6 ITE

$$\frac{eval(filter(S, e_c), e_t) \Downarrow \langle v_t, S_1 \rangle \quad eval(filter(S, \neg e_c), e_f) \Downarrow \langle v_f, S_2 \rangle}{eval(S, ITE(e_c, e_t, e_f, \mu)) \Downarrow \langle v_t \sqcup v_f, S_1 \sqcup S_2 \rangle}$$

8.7 JmpDest

$$\frac{S = \langle R, M, \langle T_M, T_C \rangle \rangle \quad t = new(T_M)}{eval(S, JmpDest(\ell, \mu)) \Downarrow \langle \langle \top, \{SymLoc(\ell, 0)\}, t \rangle, \langle R, M, \langle T_M[t \mapsto Address], T_C \rangle \rangle \rangle}$$

8.8 Load

$$\frac{eval(S, e) \Downarrow \langle v_a, S_1 \rangle \quad updateT(S_1, v_a, Address) \Downarrow S_2 \quad S_2 = \langle R_2, M_2, T_2 \rangle \quad v = \bigsqcup_{loc \in locs(v_a)} M_2(loc)}{eval(S, Load(\epsilon, k, e, \mu)) \Downarrow \langle v, S_2 \rangle}$$

8.9 Num

$$\frac{S = \langle R, M, \langle T_M, T_C \rangle \rangle \quad L = Loc(n) \quad t = new(T_M) \quad \tau = \begin{cases} Address & L \neq \emptyset \\ Value & L = \emptyset \end{cases}}{eval(S, Num(n, \mu)) \Downarrow \langle \langle n, L, t \rangle, \langle R, M, \langle T_M[t \mapsto \tau], T_C \rangle \rangle \rangle}$$

8.10 PCVar

$$\frac{S = \langle R, M, T \rangle}{eval(S, PCVar(k, name)) \Downarrow \langle R(name), S \rangle}$$

8.11 RelOp

$$\frac{\begin{array}{l} eval(S, e_1) \Downarrow \langle v_1, S_1 \rangle \quad update_{\top}(S_1, v_1, \text{Value}) \Downarrow S_2 \\ eval(S_2, e_2) \Downarrow \langle v_2, S_3 \rangle \quad update_{\top}(S_3, v_2, \text{Value}) \Downarrow S_4 \\ t = \text{new}(T_M) \quad S_4 = \langle R, M, \langle T_M, T_C \rangle \rangle \end{array}}{eval(S, \text{RelOp}(\diamond_r, e_1, e_2, \mu)) \Downarrow \langle \langle evalRelOp(\diamond_r, v_1, v_2), \emptyset, t \rangle, \langle R, M, \langle T_M[t \mapsto \text{Value}], T_C \rangle \rangle \rangle}$$

8.12 TempVar

$$\frac{S = \langle R, M, T \rangle}{eval(S, \text{TempVar}(k, id, \mu)) \Downarrow \langle R(tmp_{id}), S \rangle}$$

8.13 UnOp

$$\frac{eval(S, e) \Downarrow \langle v_1, S_1 \rangle \quad update_{\top}(S_1, v_1, \text{Value}) \Downarrow S_2 \quad evalUnOp(\diamond_u, v_1, S_2) \Downarrow \langle v_2, S_3 \rangle}{eval(S, \text{UnOp}(\diamond_u, e, \mu)) \Downarrow \langle v_2, S_3 \rangle}$$

8.14 Undefined

$$\frac{S = \langle R, M, \langle T_M, T_C \rangle \rangle \quad t = \text{new}(T_M)}{eval(S, \text{Undefined}(k, str, \mu)) \Downarrow \langle \langle \top, \emptyset, t \rangle, \langle R, M, \langle T_M[t \mapsto \top], T_C \rangle \rangle \rangle}$$

8.15 Var

$$\frac{S = \langle R, M, T \rangle}{eval(S, \text{Var}(k, reg_{id}, name, \mu)) \Downarrow \langle R(reg_{id}), S \rangle}$$

9 Statement Transfer Semantics

$$\langle l, S \rangle \rightarrow \langle l', S' \rangle$$

9.1 CJump

$$\frac{cmd(l) = \text{CJump}(e_c, e_t, e_f, \mu) \quad eval(S, e_c) \Downarrow \langle v_c, S_1 \rangle \quad update_{\top}(S_1, v_c, \text{Value}) \Downarrow S_2}{\langle l, S \rangle \rightarrow \langle e_t, filter(S_2, e_c) \rangle}$$

$$\frac{cmd(l) = \text{CJump}(e_c, e_t, e_f, \mu) \quad eval(S, e_c) \Downarrow \langle v_c, S_1 \rangle \quad update_{\top}(S_1, v_c, \text{Value}) \Downarrow S_2}{\langle l, S \rangle \rightarrow \langle e_f, filter(S_2, \neg e_c) \rangle}$$

9.2 ExternalCall

$$\frac{cmd(l) = \text{ExternalCall}(e, \mu) \quad eval(S, e) \Downarrow \langle v_f, S_1 \rangle \quad update_{\top}(S_1, v_f, \text{Address}) \Downarrow S_2 \quad f = \text{getFunction}(v_f) \quad \Sigma_{external} = \text{GetExternalSym}(f) \quad S_3 = \text{apply}(\Sigma_{external}, S_2)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S_3 \rangle}$$

9.3 IEMark

$$\frac{cmd(l) = \text{IEMark}(len, \mu)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S \rangle}$$

9.4 ISMark

$$\frac{cmd(l) = \text{ISMark}(len, \mu)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S \rangle}$$

9.5 InterCJump

$$\frac{cmd(l) = \text{InterCJump}(e_c, e_t, e_f, \mu) \quad eval(S, e_c) \Downarrow \langle v_c, S_1 \rangle \quad update_{\top}(S_1, v_c, \text{Value}) \Downarrow S_2 \quad eval(S_2, e_t) \Downarrow \langle v_t, S_3 \rangle \quad update_{\top}(S_3, v_t, \text{Address}) \Downarrow S_4}{\langle l, S \rangle \rightarrow \langle v_t, filter(S_4, e_c) \rangle}$$

$$\frac{cmd(l) = \text{InterCJump}(e_c, e_t, e_f, \mu) \quad eval(S, e_c) \Downarrow \langle v_c, S_1 \rangle \quad update_{\top}(S_1, v_c, \text{Value}) \Downarrow S_2 \quad eval(S_2, e_f) \Downarrow \langle v_f, S_3 \rangle \quad update_{\top}(S_3, v_f, \text{Address}) \Downarrow S_4}{\langle l, S \rangle \rightarrow \langle v_f, filter(S_4, \neg e_c) \rangle}$$

9.6 InterJump

$$\frac{cmd(l) = \text{InterJump}(e, jumpKind, \mu) \quad eval(S, e) \Downarrow \langle v_t, S_1 \rangle \quad update_{\top}(S_1, v_t, \text{Address}) \Downarrow S_2 \quad jumpKind.\text{IsCall} \quad f = \text{getFunction}(v_t) \quad S_3 = \text{apply}(\text{summary}(f), S_2)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S_3 \rangle}$$

$$\frac{cmd(l) = \text{InterJump}(e, jumpKind, \mu) \quad eval(S, e) \Downarrow \langle v_t, S_1 \rangle \quad update_{\top}(S_1, v_t, \text{Address}) \Downarrow S_2 \quad \neg jumpKind.\text{IsCall} \quad \neg jumpKind.\text{IsRet}}{\langle l, S \rangle \rightarrow \langle v_t, S_2 \rangle}$$

$$\frac{cmd(l) = \text{InterJump}(e, jumpKind, \mu) \quad eval(S, e) \Downarrow \langle v_r, S_1 \rangle \quad \neg jumpKind.\text{IsRet}}{\langle l, S \rangle \rightarrow \langle \text{RetAddr}, S_1 \rangle}$$

9.7 Jump

$$\frac{cmd(l) = \text{Jump}(e, \mu) \quad eval(S, e) \Downarrow \langle v_t, S_1 \rangle \quad update_{\top}(S_1, v_t, \text{Address}) \Downarrow S_2}{\langle l, S \rangle \rightarrow \langle v_t, S_2 \rangle}$$

9.8 LMark

$$\frac{cmd(l) = \text{LMark}(\ell, \mu)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S \rangle}$$

9.9 Put

$$\frac{cmd(l) = \text{Put}(e_d, e_s, \mu) \quad eval(S, e_s) \Downarrow \langle v_s, S_1 \rangle \quad S' = \text{write}(S_1, e_d, v_s)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S' \rangle}$$

$$\text{write}(\langle R, M, T \rangle, \text{Var}(k, r, n, \mu), v) = \langle R[\text{reg}_{id} \mapsto v], M, T \rangle$$

$$\text{write}(\langle R, M, T \rangle, \text{TempVar}(k, id, \mu), v) = \langle R[\text{tmp}_{id} \mapsto v], M, T \rangle$$

9.10 SideEffect

$$\frac{cmd(l) = \text{SideEffect}(\text{SysCall}, \mu) \quad S' = \text{applySysCall}(l, S)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S' \rangle}$$

$$\frac{cmd(l) = \text{SideEffect}(se, \mu)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), S \rangle}$$

9.11 Store

$$\frac{cmd(l) = \text{Store}(\epsilon, e_a, e_v, \mu) \quad eval(S, e_a) \Downarrow \langle v_a, S_1 \rangle \quad \text{update}_T(S_1, v_a, \text{Address}) \Downarrow S_2 \quad eval(S_2, e_v) \Downarrow \langle v, S_3 \rangle \quad S_3 = \langle R_3, M_3, T_3 \rangle \quad M_4 = \text{updateMem}(M_3, \text{locs}(v_a), v)}{\langle l, S \rangle \rightarrow \langle \text{next}(l), \langle R_3, M_4, T_3 \rangle \rangle}$$

$$\text{updateMem}(M, L, v) = \begin{cases} M[L \mapsto v] & |\gamma_L(L)| = 1 \\ \bigsqcup_{loc \in L} M[loc \mapsto M[loc] \sqcup v] & \text{otherwise} \end{cases}$$